Estados, Eventos y Ciclo de Renderizado

El objetivo de esta clase es dominar los **Estados, Eventos y Renderizado** en React Native para crear interfaces interactivas sin necesidad de navegación ni APIs externas.

1. Estados en React Native

El **estado** es la fuente de verdad local que determina cómo se **renderiza** un componente. Solo puede ser modificado mediante **setState** o el hook **useState**.

1.1 useState – Sintaxis y flujo

```
import React, { useState } from 'react';
import { View, Text, TextInput } from 'react-native';

export default function App() {
  const [nombre, setNombre] = useState('');
  const [edad, setEdad] = useState('');

  return (
    <View style={{ padding: 20 }}>
      <TextInput placeholder="Nombre" value={nombre}
onChangeText={setNombre} style={{ borderWidth: 1, marginBottom:
10 }} />
      <TextInput placeholder="Edad" value={edad}
onChangeText={setEdad} keyboardType="numeric" style={{
borderWidth: 1, marginBottom: 10 }} />
      <Text>{nombre && edad ? `Hola ${nombre}, tienes ${edad}
años` : 'Completa los campos'}</Text>
    </View>
  );
```

```
}
```

Concepto	¿Qué hace?
useState(0)	Declara estado inicial 0
setContador	Función para actualizar el estado y re-renderizar
Re-render	React vuelve a ejecutar el componente con nuevo valor

1.2 Buenas prácticas con estados

- Mantén el estado mínimo y **coherente**.
- Nunca mutas el estado **directamente**; siempre usa su setter.
- Agrupa estados relacionados en **objetos** o **reducers** (veremos más adelante).

2. Eventos

Los eventos en React Native son similares a los del navegador, pero con **sintaxis específica** y **gestos táctiles**.

2.1 Eventos básicos

Componente	Propiedad de evento	Descripción
`Button`	onPress	Tap simple
`Pressable`	`onPressIn` / onPressOut / onLongPress	Mayor control del gesto
TextInput	onChangeText	Cada cambio de texto

`ScrollView`	`onScroll`	Durante el desplazamiento

2.2 Ejemplo interactivo – Toggle de tema

```
import React, { useState } from 'react';
import { View, Text, Switch, StyleSheet } from 'react-native';

export default function ToggleTheme() {
  const [darkMode, setDarkMode] = useState(false);

  return (
    <View style={[styles.container, darkMode && styles.dark]}>
      <Text style={[styles.text, darkMode && styles.textDark]}>
        Modo oscuro: {darkMode ? 'ON' : 'OFF'}
      </Text>
      { /* El Switch utiliza el setDarkMode para manipular el
           valor del estado. Comprendiendo que guarda un valor
           booleano lo intercambia (false a true y true a false)
           sin funciones intermediarias */ }
      <Switch value={darkMode} onValueChange={setDarkMode} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems:
'center', backgroundColor: '#fff' },
  dark: { backgroundColor: '#121212' },
  text: { fontSize: 24, marginBottom: 16 },
  textDark: { color: '#fff' },
});
```

3. Renderizado



React Native **re-renderiza** automáticamente cuando el **estado o props** cambian. Entender **cuándo y por qué** ocurre puede mejorar el rendimiento.

3.1 Re-render vs re-mount

Escenario	¿Re-render?	¿Re-mount?
Cambio de estado (setState)	✅	❌
Cambio de props	✅	❌
Cambio de key	✅	✅
Navegación entre pantallas (más adelante)	✅ / ❌	✅ / ❌

3.2 Condicionales y listas: Renderizado condicional

```
{isLoggedIn ? (
  <Text>Bienvenido</Text>
) : (
  <Button title="Iniciar sesión" onPress={login} />
)}
```

Renderizado de listas con FlatList (básico)

Importante: *FlatList* es **lazy** – solo renderiza los elementos visibles.

```
import React, { useState } from 'react';
import { View, FlatList, Text, StyleSheet } from 'react-native';

const DATA = [
  { id: '1', title: 'Manzana' },
  { id: '2', title: 'Banana' },
  { id: '3', title: 'Cereza' },
```

```
];

export default function ListaFrutas() {
  return (
    <FlatList
      data={DATA}
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => (
        <Text style={styles.item}>{item.title}</Text>
      )}
    />
  );
}

const styles = StyleSheet.create({
  item: { padding: 16, fontSize: 20, borderBottomWidth: 1,
borderColor: '#ddd' },
});
```

4. Mini-proyecto guiado – “Lista de Tareas Simple”

1. Agregar una tarea.
2. Marcar como completada.
3. Eliminar una tarea.
4. Mostrar el contador de tareas pendientes.

Estado global (en el componente)
`tareas: Array<{id, texto, completada}>`
nuevaTarea: string

Flujo de eventos

Evento	Acción estado	Re-render

`onChangeText`	setNuevaTarea(texto)	✅
onPress (add)	`setTareas([...tareas, nuevaTarea])`	✅
onPress (toggle)	Actualizar `completada`	✅
onPress (delete)	Filtrar tareas	✅

5. Buenas prácticas rápida lista

- Usa **keys únicas** y estables en listas.
- Extrae **componentes pequeños** para evitar renders innecesarios.
- Usa **memo** (próxima clase) si un componente puro recibe props estáticas.
- Mantén la **lógica de estado** separada de la UI cuando sea posible.



Bibliografía utilizada y sugerida

Meta Platforms. React Native – Core Components and APIs.
https://reactnative.dev/docs/components-and-apis
Actualizado el 17 de junio de 2025.

Meta Platforms. React Native – Handling Text Input.
https://reactnative.dev/docs/handling-text-input
Actualizado el 17 de junio de 2025.

Meta Platforms. React Native – Using a ListView.
https://reactnative.dev/docs/using-a-listview
Actualizado el 17 de junio de 2025.

Meta Platforms. React – Hooks API Reference (useState).
https://react.dev/reference/react/useState
Actualizado el 22 de julio de 2025.

Expo Team. Expo Documentation – Managing State.
https://docs.expo.dev/tutorial/managing-state/
Actualizado el 23 de julio de 2025.

LogRocket Blog. Understanding React Native Re-renders.
https://blog.logrocket.com/understanding-react-native-re-renders
Publicado el 8 de abril de 2025.